In [1]:
```
import pandas as pd
```

In [2]:
```
df=pd.read_csv("housingData.csv")
df.head()
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.9
1	0.30701	18.0	2.99	0	0.504	6.639	64.4	1.8721	3	666	20.9	357.9
2	0.13767	80.0	19.64	0	0.692	6.788	94.9	6.9345	1	666	16.7	395.8
3	0.09729	12.5	25.13	1	0.793	6.641	37.4	2.3369	2	402	19.4	390.0
4	0.02296	0.0	6.51	0	0.503	5.831	83.5	6.0624	5	296	16.4	373.3

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    int64  
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    int64  
 9   TAX      506 non-null    int64  
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
MEDV       0
dtype: int64
```

In [5]:
```
df.shape
```

Out[5]:
```
(506, 14)
```

In [6]:
```
df.describe()
```

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	A
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000(
mean	0.106205	10.069170	14.818617	0.071146	0.601921	6.278466	64.4784
std	0.097064	22.767011	7.438254	0.257323	0.114402	0.668940	19.9067
min	0.006320	0.000000	2.120000	0.000000	0.401000	4.166000	30.0000
25%	0.033465	0.000000	7.947500	0.000000	0.510250	5.820500	46.2250
50%	0.078415	0.000000	15.250000	0.000000	0.603000	6.283000	64.4500
75%	0.146767	12.500000	21.247500	0.000000	0.700000	6.750500	82.4000
max	0.501680	80.000000	26.990000	1.000000	0.799000	8.457000	100.0000

In [7]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor,GradientBoostingRegressor
from sklearn.metrics import mean_squared_error,r2_score
import numpy as np
```

In [8]:
```
df.columns
```

Out[8]:
```
Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
       'PTRATIO', 'B', 'LSTAT', 'MEDV'],
      dtype='str')
```

In [9]:
```
X=df.drop(columns=["MEDV"])
y=df["MEDV"]
```

In [12]:
```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42
```

In [13]:
```
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
```

In [19]:
```
models={
    "Linear Regression": LinearRegression(),
    "Random Forest Regressor":RandomForestRegressor(n_estimators=100,random_stat
    "Gradient Boosting Regressor":GradientBoostingRegressor(n_estimators=100,ran
}
```

In [21]:
```
results=[]

for name, model in models.items():
    model.fit(X_train_scaled,y_train)

    predictions=model.predict(X_test_scaled)
```

In [23]:
```
rmse=np.sqrt(mean_squared_error(y_test,predictions))
r2=r2_score(y_test,predictions)

results.append({
    "Model Name": name,
    "RMSE": round(rmse,4),
    "R2 Score": round(r2,4)
})
```

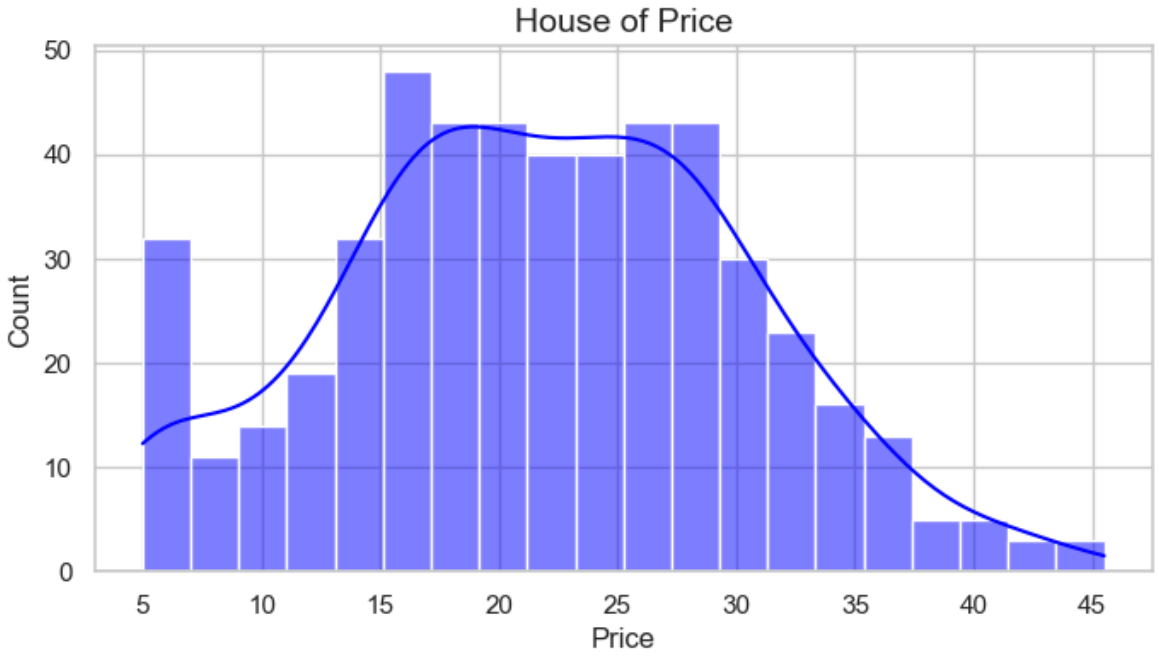
In [26]:
```
results_df=pd.DataFrame(results)
results_df
```

Out[26]:

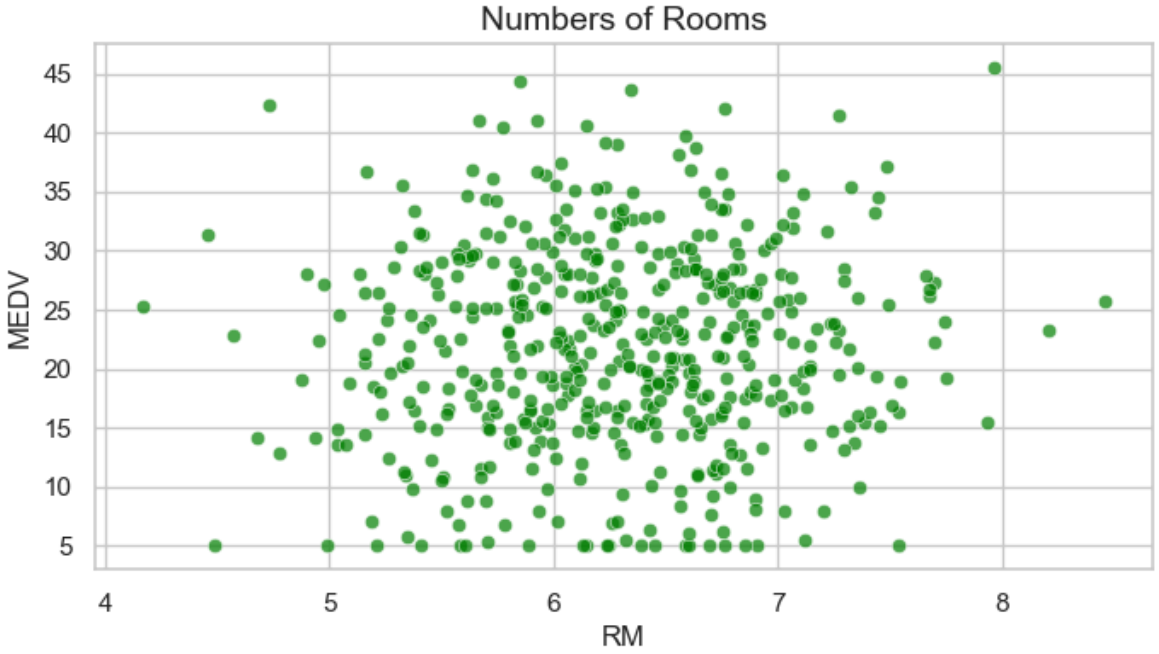
	Model Name	RMSE	R2 Score
0	Gradient Boosting Regressor	8.6313	-0.1936

In [27]:
```
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
sns.set_theme(style="whitegrid
```

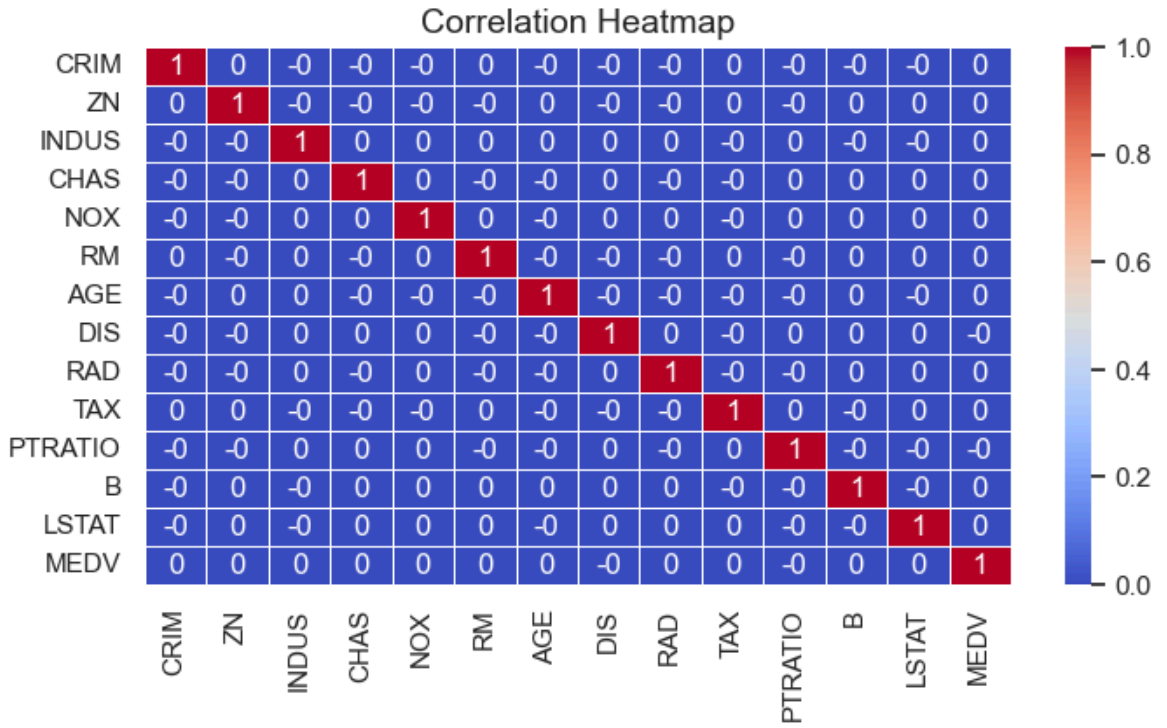
In [28]:
```
plt.figure(figsize=(8,4))
sns.histplot(df["MEDV"],kde=True,color="blue",bins=20)
plt.title("House of Price",fontsize=14)
plt.xlabel("Price",fontsize=12)
plt.ylabel("Count",fontsize=12)
plt.show()
```



In [29]:
```
plt.figure(figsize=(8,4))
sns.scatterplot(x=df["RM"],y=df["MEDV"],alpha=0.7,color="green")
plt.title("Numbers of Rooms",fontsize=14)
plt.xlabel("RM",fontsize=12)
plt.ylabel("MEDV",fontsize=12)
plt.show()
```



In [30]:
```
plt.figure(figsize=(8,4))
correlation_matrix=df.corr().round()
sns.heatmap(data=correlation_matrix,annot=True,cmap="coolwarm",linewidths=0.5)
plt.title("Correlation Heatmap",fontsize=14)
plt.show()
```


Correlation Heatmap
CRIM 1 0 -0 -0 -0 0 -0 -0 -0 0 -0 -0 -0 0
ZN 0 1 -0 -0 -0 -0 0 -0 -0 0 -0 0 0 0
INDUS -0 -0 1 0 0 0 0 0 0 -0 0 -0 -0 0
CHAS -0 -0 0 1 0 -0 -0 0 -0 -0 0 0 0 0
NOX -0 -0 0 0 1 0 -0 0 0 -0 0 0 0 0
RM 0 -0 0 -0 0 1 -0 -0 -0 0 -0 0 0 0
AGE -0 0 0 -0 -0 -0 1 -0 -0 -0 -0 0 -0 0
DIS -0 -0 0 0 0 -0 -0 1 0 -0 0 0 0 -0
RAD -0 -0 0 -0 0 -0 -0 0 1 -0 -0 0 0 0
TAX 0 0 -0 -0 -0 0 -0 -0 -0 1 0 -0 0 0
PTRATIO -0 -0 0 0 0 -0 -0 0 -0 0 1 -0 -0 -0
B -0 0 -0 0 0 0 0 0 0 -0 -0 1 -0 0
LSTAT -0 0 -0 0 0 0 -0 0 0 0 -0 -0 1 0
MEDV 0 0 0 0 0 0 0 -0 0 0 -0 0 0 1
CRIM ZN INDUS CHAS NOX RM AGE DIS RAD TAX PTRATIO B LSTAT MEDV
1.0 0.8 0.6 0.4 0.2 0.0

In [ ]: